Symbolic Waveform Design for Integrated Sensing and Communications (ISAC)

Derive the joint radar-communications tradeoff for an OFDM ISAC waveform from first principles, then validate the design with waveform generation, spectral analysis, range-Doppler processing, and EVM measurement.

Workflow: First-principles physics → symbolic ambiguity function → Pareto frontier (sensing vs. comms) → optimal pilot/data allocation → waveform generation → spectral verification → range-Doppler map → EVM

For a full system-level MIMO-OFDM ISAC simulation, see Integrated Sensing and Communication II: Communication-Centric Approach Using MIMO-OFDM.

Requires: Symbolic Math Toolbox, Communications Toolbox, Signal Processing Toolbox

1. OFDM Signal Model

An OFDM waveform partitions a wideband channel into N narrowband subcarriers spaced by Δf. In an ISAC system, some subcarriers carry known pilot symbols (used for radar sensing) and others carry random data symbols (used for communications). The split between the two is the fundamental design variable.

We define the signal model symbolically to derive the sensing and communications performance as closed-form expressions.

```
syms N positive integer          % total number of subcarriers
syms K positive integer          % number of pilot subcarriers
syms Delta_f positive            % subcarrier spacing [Hz]
syms T_sym positive              % useful OFDM symbol duration = 1/Delta_f [s]
syms T_cp positive               % cyclic prefix duration [s]
syms T positive                  % total symbol duration T_sym + T_cp [s]
syms M positive integer          % number of OFDM symbols in one frame
syms P_total positive            % total transmit power [W]
syms sigma2 positive             % noise variance (noise power) [W]
T_sym_def = 1 / Delta_f;
B = N * Delta_f;                 % total occupied bandwidth [Hz]
syms alpha positive              % alpha = K/N, pilot fraction (0,1)
P_pilot = P_total / N;          % power per pilot subcarrier
P_data  = P_total / N;          % power per data subcarrier (uniform for now)
```

2. Radar Performance: Cross-Ambiguity Function

The radar receiver correlates the received signal with the known pilot symbols to estimate target range (delay τ) and velocity (Doppler shift ν). The resolution is governed by the cross-ambiguity function, which we derive symbolically for uniformly-spaced OFDM pilots.

For K pilot subcarriers with uniform spacing N/K within a frame of M OFDM symbols, the squared ambiguity function factors into a product of frequency-domain and time-domain terms:

$$|\chi(\tau,\nu)|^2 = |A_f(\tau)|^2 \cdot |A_t(\nu)|^2$$

where A_f captures range resolution (from pilot bandwidth) and A_t captures Doppler resolution (from frame duration).

```
syms tau nu real                    % delay [s] and Doppler shift [Hz]
A_f = (1/K) * symsum(exp(-1j*2*sym(pi)*sym('k')*(N/K)*Delta_f*tau), sym('k'), 0, K-1);
A_f = simplify(A_f);
disp('Frequency-domain ambiguity A_f(tau):')
```

```
Frequency-domain ambiguity A_f(tau):
```

```
disp(A_f)
```

$$\begin{cases} 1 & \text{if } \frac{\Delta_f N \tau}{K} \in \mathbb{Z} \\ \frac{e^{-2\pi\Delta_f N\tau i} e^{\frac{2\pi\Delta_f N\tau i}{K}} \left(e^{2\pi\Delta_f N\tau i} - 1\right)}{K\left(e^{\frac{2\pi\Delta_f N\tau i}{K}} - 1\right)} & \text{if } \frac{\Delta_f N \tau}{K} \notin \mathbb{Z} \end{cases}$$

```
A_f_closed = (1/K) * (1 - exp(-1j*2*sym(pi)*N*Delta_f*tau)) / ...
             (1 - exp(-1j*2*sym(pi)*(N/K)*Delta_f*tau));
A_f_closed = simplify(A_f_closed);
disp('Closed-form (geometric series):')
```

```
Closed-form (geometric series):
```

```
disp(A_f_closed)
```

$$\frac{e^{-2\pi\Delta_f N\tau i}-1}{K\left(e^{-\frac{2\pi\Delta_f N\tau i}{K}}-1\right)}$$

Dirichlet Kernel and Range Resolution

The magnitude-squared of the frequency-domain term is a Dirichlet kernel. Using $B = N\Delta f$:

$$|A_f(\tau)|^2 = \frac{\sin^2(\pi B\tau)}{K^2\sin^2(\pi B\tau/K)}$$

The mainlobe width (range resolution) depends on the total bandwidth B, but the sidelobe structure depends on K. More pilots means lower sidelobes and better target discrimination at the cost of data throughput.

```
syms B_sym positive
A_f_mag2 = sin(sym(pi)*B_sym*tau)^2 / (K^2 * sin(sym(pi)*B_sym*tau/K)^2);
disp('|A_f(tau)|^2 (Dirichlet kernel):')
```

```
|A_f(tau)|^2 (Dirichlet kernel):
```

```
disp(A_f_mag2)
```

$$\frac{\sin(\pi\,B_{\mathrm{sym}}\,\tau)^2}{K^2\sin\left(\frac{\pi\,B_{\mathrm{sym}}\,\tau}{K}\right)^2}$$

```
syms c_light positive             % speed of light
delta_R = c_light / (2 * B_sym);
disp('Range resolution (monostatic):')
```

```
Range resolution (monostatic):
```

```
disp(delta_R)
```

$$\frac{c_{\mathrm{light}}}{2\,B_{\mathrm{sym}}}$$

Doppler Resolution

The time-domain ambiguity (Doppler resolution) comes from the coherent integration across M OFDM symbols:

$$|A_t(\nu)|^2 = \frac{\sin^2(\pi M T \nu)}{M^2 \sin^2(\pi T \nu)}$$

Doppler resolution depends on the total frame duration MT and is independent of pilot density - all M symbols contribute to Doppler processing regardless of whether they carry pilots or data.

```
A_t_mag2 = sin(sym(pi)*M*T*nu)^2 / (M^2 * sin(sym(pi)*T*nu)^2);
syms f_c positive                % carrier frequency [Hz]
delta_v = c_light / (2 * f_c * M * T);
disp('|A_t(nu)|^2:')
```

```
|A_t(nu)|^2:
```

```
disp(A_t_mag2)
```

$$\frac{\sin(\pi\ M\ T\ \nu)^2}{M^2 \sin(\pi\ T\ \nu)^2}$$

```
disp('Velocity resolution:')
```

```
Velocity resolution:
```

```
disp(delta_v)
```

$$\frac{c_{\text{light}}}{2\ M\ T\ f_c}$$

Radar SNR

The radar signal-to-noise ratio at the matched filter output depends on the number of pilot subcarriers K integrated across M symbols. For a point target at range R with radar cross section σ_{rcs}:

```
syms R_target positive           % target range [m]
```

```
syms sigma_rcs positive           % radar cross section [m^2]
syms G_ant positive               % antenna gain (linear)
syms lambda_c positive            % carrier wavelength [m]
P_r_sub = P_pilot * G_ant^2 * lambda_c^2 * sigma_rcs / ((4*sym(pi))^3 * R_target^4);
SNR_radar = K * M * P_r_sub / sigma2;
SNR_radar = simplify(SNR_radar);
disp('Radar SNR after coherent integration:')
```

```
Radar SNR after coherent integration:
```

```
disp(SNR_radar)
```

$$\frac{G_{\text{ant}}^{2}\, K\, M\, P_{\text{total}}\, \lambda_c^{2}\, \sigma_{\text{rcs}}}{64\, N\, R_{\text{target}}^{4}\, \sigma_2\, \pi^{3}}$$

```
SNR_radar_alpha = subs(SNR_radar, K, alpha*N);
SNR_radar_alpha = simplify(SNR_radar_alpha);
disp('In terms of pilot fraction alpha:')
```

```
In terms of pilot fraction alpha:
```

```
disp(SNR_radar_alpha)
```

$$\frac{G_{\text{ant}}^{2}\, M\, P_{\text{total}}\, \alpha\, \lambda_c^{2}\, \sigma_{\text{rcs}}}{64\, R_{\text{target}}^{4}\, \sigma_2\, \pi^{3}}$$

3. Communications Performance

The data subcarriers carry QAM-modulated information symbols. The achievable spectral efficiency on each data subcarrier (in the AWGN case) is given by the Shannon capacity formula. The total throughput depends on the number of data subcarriers $(1 - \alpha)N$.

Derive the per-frame throughput symbolically and express it in terms of the same pilot fraction α used in the radar analysis.

```
SNR_data = P_data / sigma2;
C_sub = log2(1 + SNR_data);
N_data = (1 - alpha) * N;
R_bits_frame = N_data * M * C_sub;
```

```
R_bits_frame = simplify(R_bits_frame);
T_frame = M * T;
R_bits_rate = R_bits_frame / T_frame;
R_bits_rate = simplify(R_bits_rate);
disp('Throughput [bits/s]:')
```

```
Throughput [bits/s]:
```

```
disp(R_bits_rate)
```

$$-\frac{N \log\left(\frac{P_{\text{total}}}{N\,\sigma_2}+1\right)(\alpha-1)}{T\log(2)}$$

```
eta_spectral = R_bits_rate / B_sym;
eta_spectral = simplify(subs(eta_spectral, B_sym, N*Delta_f));
disp('Spectral efficiency [bits/s/Hz]:')
```

```
Spectral efficiency [bits/s/Hz]:
```

```
disp(eta_spectral)
```

$$-\frac{\log\left(\frac{P_{\text{total}}}{N\,\sigma_2}+1\right)(\alpha-1)}{\Delta_f\,T\log(2)}$$

4. The ISAC Tradeoff: Pareto Frontier

Increasing the pilot fraction α improves radar SNR (linearly) but decreases communications throughput (linearly). This is the fundamental ISAC tradeoff. Normalize both metrics to their maximum achievable values (at $\alpha = 1$ for radar, $\alpha = 0$ for comms) to plot them on a common scale.

```
rho_radar = alpha;
rho_comms = 1 - alpha;
disp('Normalized radar performance:  rho_radar = alpha')
```

```
Normalized radar performance:  rho_radar = alpha
```

```
disp('Normalized comms performance:  rho_comms = 1 - alpha')
```

```
Normalized comms performance:  rho_comms = 1 - alpha
```

```
disp('Pareto constraint: rho_radar + rho_comms = 1')
```

```
Pareto constraint: rho_radar + rho_comms = 1
```

Cramér-Rao Bound for Range Estimation

The Cramér-Rao lower bound (CRB) on range estimation variance gives a more informative radar metric than raw SNR. For an OFDM radar with K uniformly-spaced pilots, the CRB on delay estimation is:

$$\mathrm{var}(\hat{\tau}) \geq \frac{3}{2\pi^2 \cdot \mathrm{SNR}_r \cdot K \cdot (K^2-1) \cdot \Delta f^2}$$

Converting to range ($R = c\tau/2$):

$$\mathrm{CRB}_R = \frac{c^2}{4} \cdot \mathrm{var}(\hat{\tau})$$

This depends on α through both $K = \alpha N$ and $\mathrm{SNR}_r \propto \alpha$.

```
CRB_tau = 3 / (2 * sym(pi)^2 * SNR_radar * K * (K^2 - 1) * Delta_f^2);
CRB_R = c_light^2 / 4 * CRB_tau;
CRB_R = simplify(CRB_R);
disp('CRB on range estimation variance:')
```

```
CRB on range estimation variance:
```

```
disp(CRB_R)
```

$$\frac{24\,\pi\,N\,{R_{\mathrm{target}}}^4\,{c_{\mathrm{light}}}^2\,\sigma_2}{{\Delta_f}^2\,{G_{\mathrm{ant}}}^2\,K^2\,M\,P_{\mathrm{total}}\,{\lambda_c}^2\,\sigma_{\mathrm{rcs}}\,(K^2-1)}$$

```
CRB_R_alpha = subs(CRB_R, K, alpha*N);
CRB_R_alpha = simplify(CRB_R_alpha);
disp('In terms of alpha:')
```

```
In terms of alpha:
```

```
disp(CRB_R_alpha)
```

$$\frac{24\,\pi\,R_{\text{target}}{}^4\,c_{\text{light}}{}^2\,\sigma_2}{\Delta_f{}^2\,G_{\text{ant}}{}^2\,M\,N\,P_{\text{total}}\,\alpha^2\,\lambda_c{}^2\,\sigma_{\text{rcs}}\,(N^2\,\alpha^2-1)}$$

```
RCRB = sqrt(CRB_R_alpha);
disp('Range RMSE lower bound (root CRB):')
```

```
Range RMSE lower bound (root CRB):
```

```
disp(RCRB)
```

$$\frac{2\,\sqrt{6}\,R_{\text{target}}{}^2\,c_{\text{light}}\,\sqrt{\sigma_2}\,\sqrt{\pi}\,\sqrt{\frac{1}{N^2\,\alpha^2-1}}}{\Delta_f\,G_{\text{ant}}\,\sqrt{M}\,\sqrt{N}\,\sqrt{P_{\text{total}}}\,\alpha\,\lambda_c\,\sqrt{\sigma_{\text{rcs}}}}$$

Pareto Visualization

Evaluate the ISAC tradeoff at a concrete design point and generate the Pareto frontier, showing how pilot fraction α trades radar range accuracy against communications throughput.

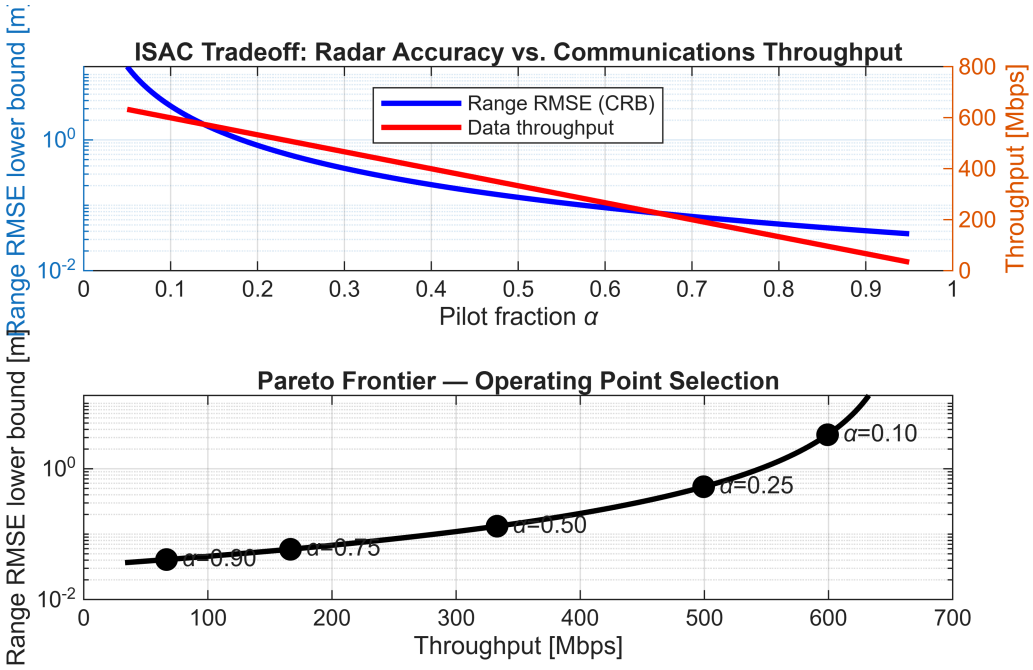
```
N_val       = 1024;               % subcarriers
Delta_f_val = 30e3;               % 30 kHz subcarrier spacing
T_cp_val    = 1/(Delta_f_val*14); % ~2.38 us CP (normal CP at 30 kHz)
T_val       = 1/Delta_f_val + T_cp_val;
M_val       = 14;                 % 14 symbols per slot (one 5G slot)
P_total_val = 1;                  % 1 W transmit power
sigma2_val  = 1e-10;              % noise power [W]
f_c_val     = 3.5e9;              % 3.5 GHz carrier
c_val       = 3e8;                % speed of light
lambda_val  = c_val / f_c_val;
G_ant_val   = 10^(15/10);         % 15 dBi antenna gain
R_target_val = 200;               % 200 m target range
sigma_rcs_val = 10;               % 10 m^2 RCS (vehicle)
cal_vars = [N, Delta_f, T_cp, T, M, P_total, sigma2, f_c, c_light, ...
            lambda_c, G_ant, R_target, sigma_rcs];
```

```
cal_vals = [N_val, Delta_f_val, T_cp_val, T_val, M_val, P_total_val, ...
            sigma2_val, f_c_val, c_val, lambda_val, G_ant_val, ...
            R_target_val, sigma_rcs_val];
CRB_R_fun = matlabFunction(subs(CRB_R_alpha, cal_vars, cal_vals), 'Vars', {alpha});
R_rate_fun = matlabFunction(subs(R_bits_rate, [cal_vars, B_sym], ...
    [cal_vals, N_val*Delta_f_val]), 'Vars', {alpha});
alpha_sweep = linspace(0.05, 0.95, 500);
CRB_sweep   = arrayfun(CRB_R_fun, alpha_sweep);
RCRB_sweep  = sqrt(CRB_sweep);
rate_sweep  = arrayfun(R_rate_fun, alpha_sweep);
```

ISAC Tradeoff Curves

```
tiledlayout(2,1)
nexttile
yyaxis left
plot(alpha_sweep, RCRB_sweep, 'b-', 'LineWidth', 2)
ylabel('Range RMSE lower bound [m]')
set(gca, 'YScale', 'log')
yyaxis right
plot(alpha_sweep, rate_sweep/1e6, 'r-', 'LineWidth', 2)
ylabel('Throughput [Mbps]')
xlabel('Pilot fraction \alpha')
title('ISAC Tradeoff: Radar Accuracy vs. Communications Throughput')
legend('Range RMSE (CRB)', 'Data throughput', 'Location', 'north')
grid on
nexttile
plot(rate_sweep/1e6, RCRB_sweep, 'k-', 'LineWidth', 2)
hold on
alpha_pts = [0.1, 0.25, 0.5, 0.75, 0.9];
for i = 1:length(alpha_pts)
    crb_pt = sqrt(CRB_R_fun(alpha_pts(i)));
    rate_pt = R_rate_fun(alpha_pts(i))/1e6;
    plot(rate_pt, crb_pt, 'ko', 'MarkerSize', 8, 'MarkerFaceColor', 'k')
    text(rate_pt, crb_pt, sprintf('  \\alpha=%.2f', alpha_pts(i)), 'FontSize', 9)
end
hold off
```

```
xlabel('Throughput [Mbps]')
ylabel('Range RMSE lower bound [m]')
title('Pareto Frontier — Operating Point Selection')
set(gca, 'YScale', 'log')
grid on
```



Design Point Summary

```
fprintf('Bandwidth:       %.1f MHz\n', N_val*Delta_f_val/1e6)
```

```
Bandwidth:       30.7 MHz
```

```
fprintf('Frame duration:  %.2f ms\n', M_val*T_val*1e3)
```

```
Frame duration:  0.50 ms
```

```
fprintf('Carrier freq:    %.1f GHz\n', f_c_val/1e9)
```

```
Carrier freq:    3.5 GHz
```

```
fprintf('Target range:    %d m, RCS = %d m^2\n', R_target_val, sigma_rcs_val)
```

```
Target range:    200 m, RCS = 10 m^2
```

5. Pilot Sequence Design with Number-Theoretic Constraints

For the pilot subcarriers, we use Zadoff-Chu (ZC) sequences because of their ideal periodic autocorrelation, which is critical for unambiguous radar range estimation. ZC sequences have two number-theoretic requirements:

1. The sequence length L must be prime (for zero periodic cross-correlation between different root indices)
2. The root index u must be coprime with L (automatically satisfied when L is prime and $1 \le u < L$)

Given a desired number of pilot subcarriers $K = \alpha N$, we must find the nearest valid (prime) sequence length. This is where `prevprime` and `nextprime` naturally enter the workflow.

```
alpha_sense = sym(3)/4;           % sensing-heavy: 75% pilots
alpha_comms = sym(1)/4;           % comms-heavy: 25% pilots
K_sense_raw = alpha_sense * N_val;
K_comms_raw = alpha_comms * N_val;
fprintf('Sensing-heavy (alpha=0.75): K_raw = %d\n', K_sense_raw)
```

```
Sensing-heavy (alpha=0.75): K_raw = 768
```

```
fprintf('  Next prime:     L = %s\n', char(nextprime(K_sense_raw)))
```

```
  Next prime:     L = 769
```

```
fprintf('  Previous prime: L = %s\n', char(prevprime(K_sense_raw)))
```

```
  Previous prime: L = 761
```

```
fprintf('Comms-heavy (alpha=0.25):   K_raw = %d\n', K_comms_raw)
```

```
Comms-heavy (alpha=0.25):   K_raw = 256
```

```
fprintf('  Next prime:     L = %s\n', char(nextprime(K_comms_raw)))
```

```
  Next prime:     L = 257
```

```
fprintf('  Previous prime: L = %s\n', char(prevprime(K_comms_raw)))
```

```
  Previous prime: L = 251
```

```
L_sense = double(prevprime(K_sense_raw));
L_comms = double(nextprime(K_comms_raw));
fprintf('Selected pilot lengths:\n')
```

```
Selected pilot lengths:
```

```
fprintf('  Sensing-heavy: L = %d (prime)\n', L_sense)
```

```
  Sensing-heavy: L = 761 (prime)
```

```
fprintf('  Comms-heavy:   L = %d (prime)\n', L_comms)
```

```
  Comms-heavy:   L = 257 (prime)
```

Zadoff-Chu Root Index Selection

The root index u determines the slope of the ZC sequence's frequency chirp, which directly affects the delay-Doppler coupling in the ambiguity function. Different roots u and u' produce zero cross-correlation when L is prime, enabling multi-user ISAC or MIMO radar. Verify co-primality symbolically.

```
syms u positive integer
syms L_zc positive integer
syms n_idx integer
zc_seq = exp(-1j * sym(pi) * u * n_idx * (n_idx + 1) / L_zc);
disp('Zadoff-Chu sequence:')
```

```
Zadoff-Chu sequence:
```

```
disp(zc_seq)
```

$$\mathrm{e}^{-\frac{\pi\, n_{\mathrm{idx}}\, u\, \left(n_{\mathrm{idx}}+1\right)\,\mathrm{i}}{L_{\mathrm{zc}}}}$$

```
u_candidate = sym(7);
L_check = sym(L_sense);
```

```
g = gcd(u_candidate, L_check);
fprintf('Root index u = %s, length L = %s\n', char(u_candidate), char(L_check))
```

```
Root index u = 7, length L = 761
```

```
fprintf('gcd(u, L) = %s\n', char(g))
```

```
gcd(u, L) = 1
```

```
if g == 1
    fprintf('Coprime: valid ZC root\n')
end
```

```
Coprime: valid ZC root
```

```
fprintf('Number of valid roots for L = %d: %d (all of 1..%d)\n', ...
    L_sense, L_sense-1, L_sense-1)
```

```
Number of valid roots for L = 761: 760 (all of 1..760)
```

Autocorrelation Verification

The periodic autocorrelation of a ZC sequence is an ideal impulse. Verify this symbolically by computing the correlation sum, then confirm numerically with `zadoffChuSeq`.

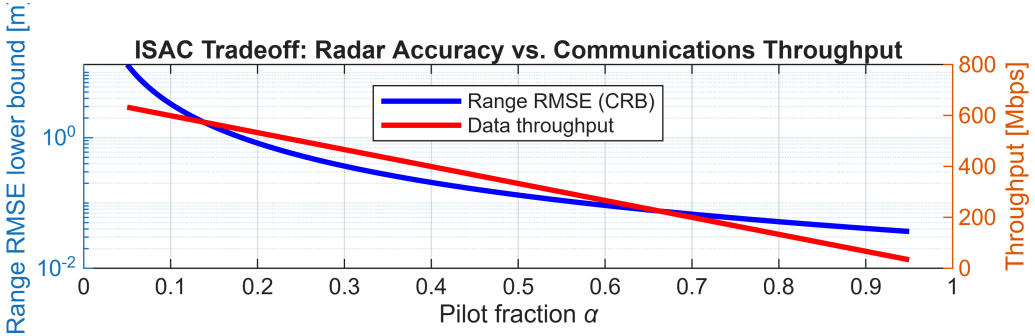
```
syms m_shift integer
R_auto = symsum(exp(-1j*sym(pi)*u*n_idx*(n_idx+1)/L_zc) * ...
    exp(1j*sym(pi)*u*(n_idx+m_shift)*(n_idx+m_shift+1)/L_zc), n_idx, 0, L_zc-1);
R_auto_simplified = simplify(R_auto);
disp('Periodic autocorrelation R(m) (symbolic):')
```

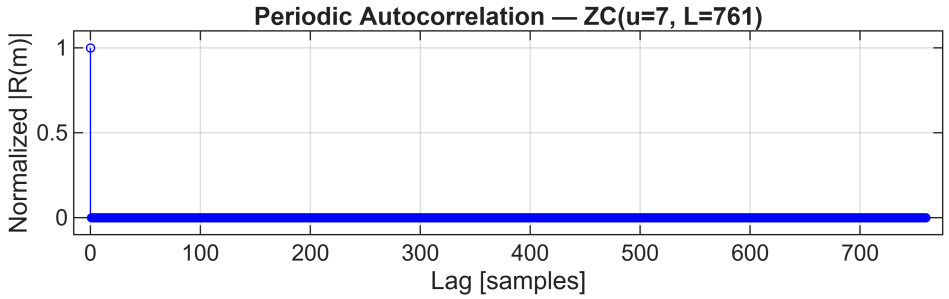
```
Periodic autocorrelation R(m) (symbolic):
```

```
disp(R_auto_simplified)
```

$$\begin{cases} L_{zc}\, e^{\frac{\pi\, m_{shift}\, u\, \left(m_{shift}\, i + i\right)}{L_{zc}}} & \text{if } \frac{m_{shift}\, u}{L_{zc}} \in \mathbb{Z} \\ -\frac{e^{\frac{\pi\, m_{shift}^{2}\, u\, i}{L_{zc}}}\, e^{\frac{\pi\, m_{shift}\, u\, i}{L_{zc}}} - e^{\frac{\pi\, m_{shift}^{2}\, u\, i}{L_{zc}}}\, e^{\frac{\pi\, m_{shift}\, u\, i}{L_{zc}}}\, e^{2\,\pi\, m_{shift}\, u\, i}}{e^{\frac{2\,\pi\, m_{shift}\, u\, i}{L_{zc}}} - 1} & \text{if } \frac{m_{shift}\, u}{L_{zc}} \notin \mathbb{Z} \end{cases}$$

```
u_val = 7;
zc_sense = zadoffChuSeq(u_val, L_sense);
R_numeric = abs(ifft(fft(zc_sense) .* conj(fft(zc_sense))));
R_numeric = R_numeric / max(R_numeric);
stem(0:L_sense-1, R_numeric, 'b', 'MarkerSize', 3)
xlabel('Lag [samples]')
ylabel('Normalized |R(m)|')
title(sprintf('Periodic Autocorrelation — ZC(u=%d, L=%d)', u_val, L_sense))
ylim([-0.1 1.1])
grid on
```






```
fprintf('Peak sidelobe level: %.2e (ideally 0)\n', max(R_numeric(2:end)))
```

```
Peak sidelobe level: 1.81e-13 (ideally 0)
```

6. Convert Symbolic Design to Numerical Simulation

We now have two operating points on the Pareto frontier, each with a valid ZC pilot sequence length. The symbolic analysis produced:

- Function handles via `matlabFunction`: CRB and throughput vs. α
- Scalar parameters via `double`: L, u, Δf, T_{cp}, M

Generate the actual OFDM ISAC waveform at each operating point and compare the predicted vs. measured performance.

```
K1          = L_sense;           % use the prime-length pilot count
N_data1     = N_val - K1;
alpha1_actual = K1 / N_val;
K2          = L_comms;
```

```
N_data2     = N_val - K2;
alpha2_actual = K2 / N_val;
fprintf('Point 1 — Sensing-heavy:\n')
```

```
Point 1 — Sensing-heavy:
```

```
fprintf('  Pilots: K = %d (ZC length, prime)\n', K1)
```

```
  Pilots: K = 761 (ZC length, prime)
```

```
fprintf('  Data:   %d subcarriers\n', N_data1)
```

```
  Data:   263 subcarriers
```

```
fprintf('  alpha:  %.4f\n', alpha1_actual)
```

```
  alpha:  0.7432
```

```
fprintf('  Predicted range RMSE: %.3f m\n', sqrt(CRB_R_fun(alpha1_actual)))
```

```
  Predicted range RMSE: 0.060 m
```

```
fprintf('  Predicted throughput: %.1f Mbps\n', R_rate_fun(alpha1_actual)/1e6)
```

```
  Predicted throughput: 171.0 Mbps
```

```
fprintf('Point 2 — Comms-heavy:\n')
```

```
Point 2 — Comms-heavy:
```

```
fprintf('  Pilots: K = %d (ZC length, prime)\n', K2)
```

```
  Pilots: K = 257 (ZC length, prime)
```

```
fprintf('  Data:   %d subcarriers\n', N_data2)
```

```
  Data:   767 subcarriers
```

```
fprintf('  alpha:  %.4f\n', alpha2_actual)
```

```
  alpha:  0.2510
```

```
fprintf('  Predicted range RMSE: %.3f m\n', sqrt(CRB_R_fun(alpha2_actual)))
```

```
Predicted range RMSE: 0.525 m
```

```
fprintf('  Predicted throughput: %.1f Mbps\n', R_rate_fun(alpha2_actual)/1e6)
```

```
Predicted throughput: 498.7 Mbps
```

Waveform Generation

Generate one frame of the OFDM ISAC waveform at each operating point. Pilot subcarriers carry the Zadoff-Chu sequence with a per-symbol pseudo-random phase scramble to whiten the spectrum (avoiding periodic repetition artifacts). Data subcarriers carry QPSK symbols. The scrambling is transparent to both the radar processor (which divides out the known transmitted grid) and the communications equalizer (which estimates the channel from the known pilots).

```
rng(42)
mod_order = 4;  % QPSK
[grid1, pilot_idx1, data_idx1, zc1, data1] = ...
    buildISACGrid(N_val, K1, L_sense, u_val, M_val, mod_order);
[grid2, pilot_idx2, data_idx2, zc2, data2] = ...
    buildISACGrid(N_val, K2, L_comms, u_val, M_val, mod_order);
nfft = N_val;
cp_len = round(T_cp_val * N_val * Delta_f_val);
tx_waveform1 = ofdmmod(grid1, nfft, cp_len);
tx_waveform2 = ofdmmod(grid2, nfft, cp_len);
fprintf('OFDM symbols per frame: %d\n', M_val)
```

```
OFDM symbols per frame: 14
```

```
fprintf('FFT size: %d,  CP length: %d samples\n', nfft, cp_len)
```

```
FFT size: 1024,  CP length: 73 samples
```

```
fprintf('Waveform 1 (sensing): %d samples\n', length(tx_waveform1))
```

```
Waveform 1 (sensing): 15358 samples
```

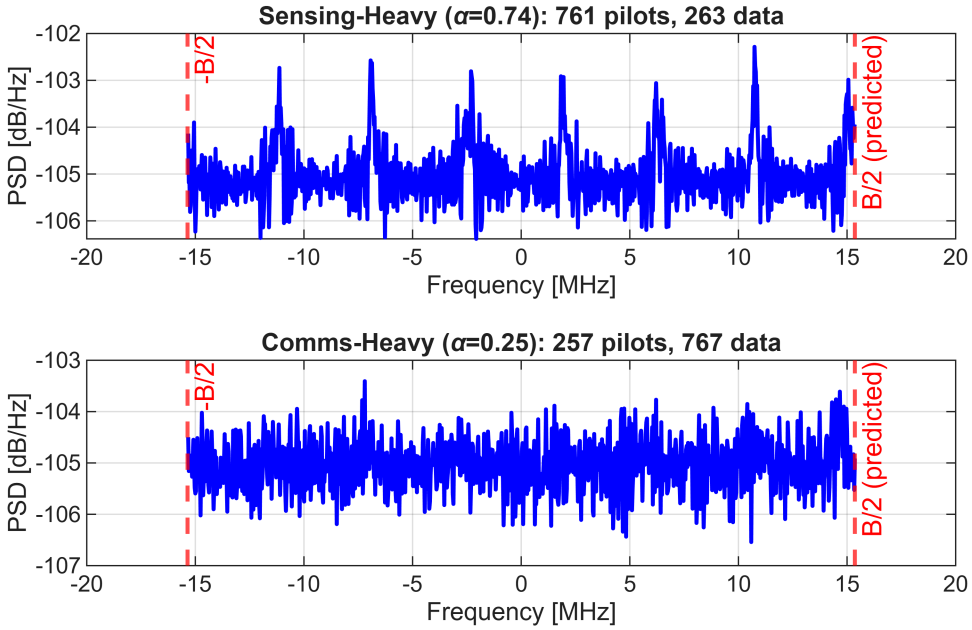
```
fprintf('Waveform 2 (comms):   %d samples\n', length(tx_waveform2))
```

```
Waveform 2 (comms):   15358 samples
```

7. Spectral Verification

Verify that the generated waveform's power spectral density matches the symbolic bandwidth prediction $B = N\Delta f$.

```
Fs = N_val * Delta_f_val;
B_predicted = N_val * Delta_f_val;
tiledlayout(2,1)
nexttile
[psd1, f1] = pwelch(tx_waveform1, hamming(nfft), nfft/2, nfft, Fs, 'centered');
plot(f1/1e6, pow2db(psd1), 'b-', 'LineWidth', 1.5)
hold on
xline(B_predicted/2e6, 'r--', 'B/2 (predicted)', 'LineWidth', 1.5)
xline(-B_predicted/2e6, 'r--', '-B/2', 'LineWidth', 1.5)
hold off
xlabel('Frequency [MHz]')
ylabel('PSD [dB/Hz]')
title(sprintf('Sensing-Heavy (\\alpha=%.2f): %d pilots, %d data', ...
    alpha1_actual, K1, N_data1))
grid on
nexttile
[psd2, f2] = pwelch(tx_waveform2, hamming(nfft), nfft/2, nfft, Fs, 'centered');
plot(f2/1e6, pow2db(psd2), 'b-', 'LineWidth', 1.5)
hold on
xline(B_predicted/2e6, 'r--', 'B/2 (predicted)', 'LineWidth', 1.5)
xline(-B_predicted/2e6, 'r--', '-B/2', 'LineWidth', 1.5)
hold off
xlabel('Frequency [MHz]')
ylabel('PSD [dB/Hz]')
title(sprintf('Comms-Heavy (\\alpha=%.2f): %d pilots, %d data', ...
    alpha2_actual, K2, N_data2))
grid on
```




```
fprintf('Predicted bandwidth: %.2f MHz\n', B_predicted/1e6)
```

```
Predicted bandwidth: 30.72 MHz
```

8. Range-Doppler Processing

Simulate a single target at known range and velocity, then recover it from the OFDM radar processing chain:

1. OFDM-demodulate the received signal
2. Element-wise divide by the known pilot symbols (matched filter)
3. IFFT across subcarriers (range axis)
4. FFT across OFDM symbols (Doppler axis)

Process both operating points to show how pilot density affects the range-Doppler map quality.

```
target_range = 150;              % [m]
target_velocity = 30;            % [m/s]
```

```
target_delay = 2 * target_range / c_val;
target_doppler = 2 * target_velocity * f_c_val / c_val;
fprintf('Range:     %d m  (delay = %.3f us)\n', target_range, target_delay*1e6)
```

```
Range:     150 m  (delay = 1.000 us)
```

```
fprintf('Velocity:  %d m/s (Doppler = %.1f Hz)\n', target_velocity, target_doppler)
```

```
Velocity:  30 m/s (Doppler = 700.0 Hz)
```

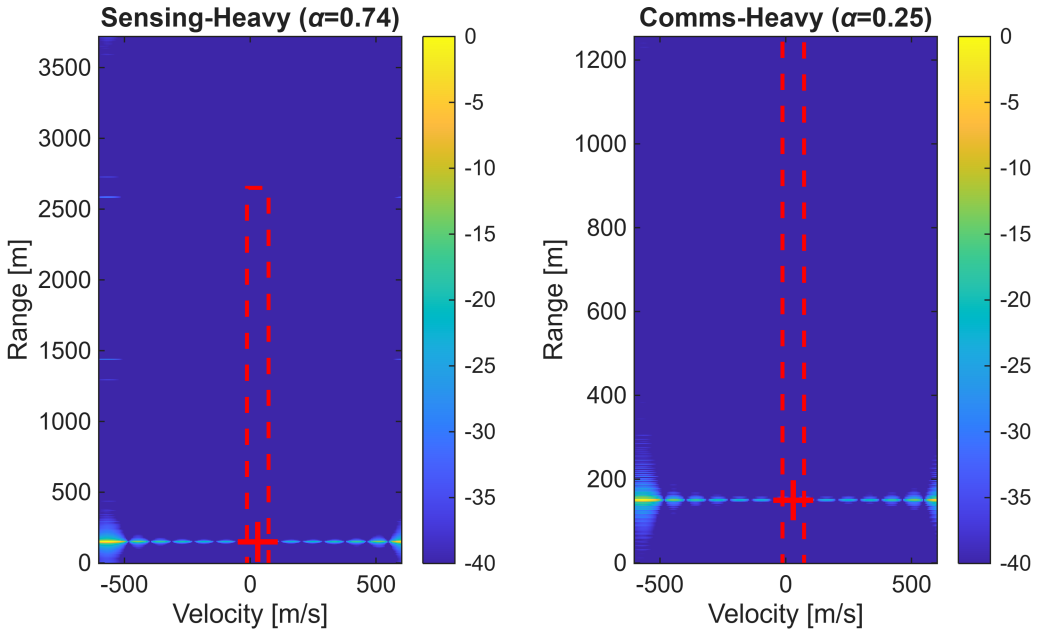
```
snr_db = 20;
rx_grid1 = applyTargetChannel(grid1, N_val, M_val, ...
    target_delay, target_doppler, Delta_f_val, T_val, snr_db);
rx_grid2 = applyTargetChannel(grid2, N_val, M_val, ...
    target_delay, target_doppler, Delta_f_val, T_val, snr_db);
nfft_range = 512;
nfft_doppler = 256;
[rd1, range_ax1, vel_ax1] = ofdmRadarProcess(rx_grid1, grid1, ...
    pilot_idx1, N_val, M_val, Delta_f_val, T_val, c_val, f_c_val, ...
    nfft_range, nfft_doppler);
[rd2, range_ax2, vel_ax2] = ofdmRadarProcess(rx_grid2, grid2, ...
    pilot_idx2, N_val, M_val, Delta_f_val, T_val, c_val, f_c_val, ...
    nfft_range, nfft_doppler);
```

Range-Doppler Maps

The red cross marks the true target location. The dashed rectangle shows the resolution cell predicted by the symbolic ambiguity function derivation.

```
delta_v_pred = c_val / (2 * f_c_val * M_val * T_val);
delta_R_pred1 = c_val / (2 * K1 * (N_val/K1) * Delta_f_val) * N_val;
delta_R_pred2 = c_val / (2 * K2 * (N_val/K2) * Delta_f_val) * N_val;
tiledlayout(1,2)
nexttile
imagesc(vel_ax1, range_ax1, pow2db(rd1 / max(rd1(:))))
axis xy
colorbar
clim([-40 0])
hold on
plot(target_velocity, target_range, 'r+', 'MarkerSize', 15, 'LineWidth', 2)
```

```
rectangle('Position', [target_velocity-delta_v_pred/2, target_range-delta_R_pred1/2, ...
    delta_v_pred, delta_R_pred1], 'EdgeColor', 'r', 'LineWidth', 1.5, 'LineStyle', '--')
hold off
xlabel('Velocity [m/s]')
ylabel('Range [m]')
title(sprintf('Sensing-Heavy (\\alpha=%.2f)', alpha1_actual))
nexttile
imagesc(vel_ax2, range_ax2, pow2db(rd2 / max(rd2(:))))
axis xy
colorbar
clim([-40 0])
hold on
plot(target_velocity, target_range, 'r+', 'MarkerSize', 15, 'LineWidth', 2)
rectangle('Position', [target_velocity-delta_v_pred/2, target_range-delta_R_pred2/2, ...
    delta_v_pred, delta_R_pred2], 'EdgeColor', 'r', 'LineWidth', 1.5, 'LineStyle', '--')
hold off
xlabel('Velocity [m/s]')
ylabel('Range [m]')
title(sprintf('Comms-Heavy (\\alpha=%.2f)', alpha2_actual))
```



9. EVM Measurement for Communications Quality

The Error Vector Magnitude (EVM) quantifies how accurately the data symbols are recovered. In a mobile channel, Doppler shift causes phase rotation across OFDM symbols/ So without compensation, the constellation "spins" and EVM is very high.

The same pilots used for radar sensing also provide per-symbol channel estimates for communications equalization. Estimate the channel on the pilot subcarriers, interpolate to the data subcarriers, and equalize. More pilots improves both radar accuracy and equalization quality.

```
eq_data1 = pilotEqualize(rx_grid1, grid1, pilot_idx1, data_idx1);
eq_data2 = pilotEqualize(rx_grid2, grid2, pilot_idx2, data_idx2);
ref_constellation = qammod(0:mod_order-1, mod_order, 'UnitAveragePower', true);
evm_meter = comm.EVM('ReferenceSignalSource', 'Estimated from reference constellation', ...
    'ReferenceConstellation', ref_constellation);
eq_data1_vec = eq_data1(:);
evm1 = evm_meter(eq_data1_vec);
release(evm_meter);
eq_data2_vec = eq_data2(:);
```

```
evm2 = evm_meter(eq_data2_vec);
fprintf('Sensing-heavy (alpha=%.2f): EVM = %.2f%%n', alpha1_actual, evm1)
```

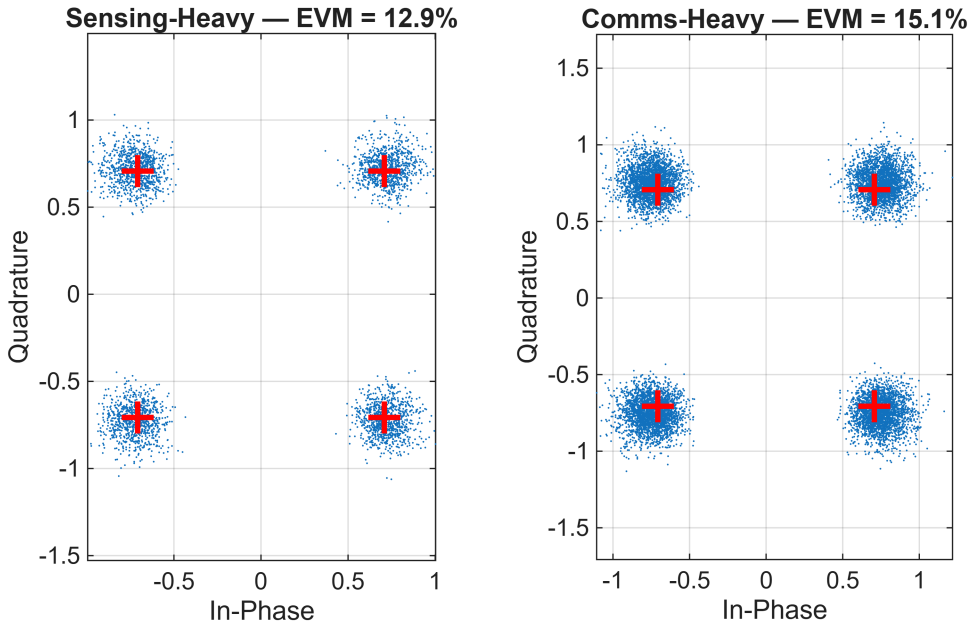
```
Sensing-heavy (alpha=0.74): EVM = 12.88%n
```

```
fprintf('Comms-heavy   (alpha=%.2f): EVM = %.2f%%n', alpha2_actual, evm2)
```

```
Comms-heavy   (alpha=0.25): EVM = 15.15%n
```

Received Constellations

```
tiledlayout(1,2)
nexttile
plot(eq_data1_vec, '.', 'MarkerSize', 2)
hold on
plot(ref_constellation, 'r+', 'MarkerSize', 12, 'LineWidth', 2)
hold off
axis equal; grid on
title(sprintf('Sensing-Heavy — EVM = %.1f%%', evm1))
xlabel('In-Phase'); ylabel('Quadrature')
nexttile
plot(eq_data2_vec, '.', 'MarkerSize', 2)
hold on
plot(ref_constellation, 'r+', 'MarkerSize', 12, 'LineWidth', 2)
hold off
axis equal; grid on
title(sprintf('Comms-Heavy — EVM = %.1f%%', evm2))
xlabel('In-Phase'); ylabel('Quadrature')
```



Local Functions

```
function [grid, pilot_idx, data_idx, zc_pilots, data_syms] = ...
        buildISACGrid(N, K_pilots, L_zc, u_root, M_sym, mod_order)
    pilot_idx = round(linspace(1, N, K_pilots));
    pilot_idx = unique(pilot_idx);
    K_pilots = length(pilot_idx);
    all_idx = 1:N;
    data_idx = setdiff(all_idx, pilot_idx);
    zc_full = zadoffChuSeq(u_root, L_zc);
    zc_pilots = zc_full(1:K_pilots);
    data_bits = randi([0 mod_order-1], length(data_idx)*M_sym, 1);
    data_syms_all = qammod(data_bits, mod_order, 'UnitAveragePower', true);
    data_syms = reshape(data_syms_all, length(data_idx), M_sym);
    grid = zeros(N, M_sym);
    scramble_phases = exp(1j*2*pi*rand(1, M_sym));
    grid(pilot_idx, :) = zc_pilots .* scramble_phases;
```

```
    grid(data_idx, :) = data_syms;
end
function eq_data = pilotEqualize(rx_grid, tx_grid, pilot_idx, data_idx)
    H_pilots = rx_grid(pilot_idx, :) ./ tx_grid(pilot_idx, :);
    H_all = interp1(pilot_idx(:), H_pilots, (1:size(rx_grid,1)).', 'linear', 'extrap');
    eq_grid = rx_grid ./ H_all;
    eq_data = eq_grid(data_idx, :);
end
function rx_grid = applyTargetChannel(tx_grid, N, M_sym, ...
        delay, doppler, Delta_f, T_total, snr_db)
    rx_grid = zeros(size(tx_grid));
    for m = 1:M_sym
        for k = 1:N
            phase_delay = exp(-1j*2*pi*(k-1)*Delta_f*delay);
            phase_doppler = exp(1j*2*pi*doppler*(m-1)*T_total);
            rx_grid(k, m) = tx_grid(k, m) * phase_delay * phase_doppler;
        end
    end
    noise_power = 10^(-snr_db/10) * mean(abs(rx_grid(:)).^2);
    noise = sqrt(noise_power/2) * (randn(size(rx_grid)) + 1j*randn(size(rx_grid)));
    rx_grid = rx_grid + noise;
end
function [rd_map, range_axis, vel_axis] = ofdmRadarProcess(...
        rx_grid, tx_grid, pilot_idx, N, M_sym, Delta_f, T_total, c, fc, nfft_r, nfft_d)
    rx_pilots = rx_grid(pilot_idx, :);
    tx_pilots = tx_grid(pilot_idx, :);
    H_est = rx_pilots ./ tx_pilots;
    rd_map = fft(ifft(H_est, nfft_r, 1), nfft_d, 2);
    rd_map = abs(rd_map).^2;
    K_p = length(pilot_idx);
    max_range = c / (2 * (N/K_p) * Delta_f);
    range_axis = linspace(0, max_range, nfft_r);
    max_doppler = 1 / (2 * T_total);
    doppler_axis = linspace(-max_doppler, max_doppler, nfft_d);
    vel_axis = doppler_axis * c / (2 * fc);
end
```